



Section 7 / 24

Go to Questions ↓

Assembling & Disassembling

Now that we understand the basic structure and elements of an Assembly file, we can start assembling it using the `nasm` tool. The entire assembly file structure we learned in the previous section is based on the `nasm` file structure. Upon assembling our code with `nasm`, it understands the various parts of the file and then correctly assembles them to be properly run during run time.

After we assemble our code with `nasm`, we can link it using `ld` to utilize various OS features and libraries.

Assembling

First, we will copy the code below into a file called `helloWorld.s`.

Note: assembly files usually use the `.s` or the `.asm` extensions. We will be using `.s` in this module.

We don't have to keep using tabs to separate parts of an assembly file, as this was only for demonstration purposes. We can write the following code into our `helloWorld.s` file:

```
global _start

section .data
    message db "Hello HTB Academy!"
    length equ $-message

section .text
_start:
    mov rax, 1
    mov rdi, 1
    mov rsi, message
    mov rdx, length
    syscall

    mov rax, 60
    mov rdi, 0
    syscall
```

Note how we used `equ` to dynamically calculate the length of `message`, instead of using a static `18`. This will become very handy later on. Once we do, we will assemble the file using `nasm`, with the following command:

```
kimvutrinh@htb[/htb]$ nasm -f elf64 helloWorld.s
```

Note: The `-f elf64` flag is used to note that we want to assemble a 64-bit assembly code. If we wanted to assemble a 32-bit code, we would use `-f elf`.

This should output a `helloWorld.o` object file, which is then assembled into machine code, along with the details of all variables and sections. This file is not executable just yet.

Linking

The final step is to link our file using `ld`. The `helloWorld.o` object file, though assembled, still cannot be executed. This is because many references and labels used by `nasm` need to be resolved into actual addresses, along with linking the file with various OS libraries that may be needed.

This is why a Linux binary is called `ELF`, which stands for an `Executable and Linkable Format`. To link a file using `ld`, we can use the following command:

```
shellsession
kimvutrinh@htb[/htb]$ ld -o helloWorld helloWorld.o
```

Note: if we were to assemble a 32-bit binary, we need to add the '`-m elf_i386`' flag.

Once we link the file with `ld`, we should have the final executable file:

```
shellsession
kimvutrinh@htb[/htb]$ ./helloWorld
Hello HTB Academy!
```

We have successfully assembled and linked our first assembly file. We will be assembling, linking, and running our code frequently through this module, so let us build a simple `bash` script to make it easier:

```
#!/bin/bash
```

```
fileName="${1%.*}" # remove .s extension
```

```
nasm -f elf64 ${fileName}.s
```

```
ld ${fileName}.o -o ${fileName}
```

```
[ "$2" == "-g" ] && gdb -q ${fileName} || ./${fileName}
```

Now we can write this script to `assembler.sh`, `chmod +x` it, and then run it on our assembly file. It will assemble it, link it, and run it:

```
kimvutrinh@htb[/htb]$ ./assembler.sh helloWorld.s
```

```
Hello HTB Academy!
```

Great! Before we move on, let's disassemble and examine our files to learn more about the process we just did.

Disassembling

To disassemble a file, we will use the `objdump` tool, which dumps machine code from a file and interprets the assembly instruction of each hex code. We can disassemble a binary using the `-D` flag.

Note: we will also use the flag `-M intel`, so that `objdump` would write the instructions in the Intel syntax, which we are using, as we discussed before.

Let's start by disassembling our final ELF executable file:

shellsession

```
kimvutrinh@htb[/htb]$ objdump -M intel -d helloWorld
```

```
helloWorld:      file format elf64-x86-64
```

```
Disassembly of section .text:
```

```
0000000000401000 <_start>:
```

```
401000:  b8 01 00 00 00      mov     eax,0x1
401005:  bf 01 00 00 00      mov     edi,0x1
40100a:  48 be 00 20 40 00 00 movabs  rsi,0x402000
401011:  00 00 00
401014:  ba 12 00 00 00      mov     edx,0x12
401019:  0f 05               syscall
40101b:  b8 3c 00 00 00      mov     eax,0x3c
401020:  bf 00 00 00 00      mov     edi,0x0
401025:  0f 05               syscall
```

We see that our original assembly code is highly preserved, with the only change being `0x402000` used in place of the `message` variable and replacing the `length` constant with its value of `0x12`. We also see that `nasm` efficiently changed our `64-bit` registers to the `32-bit` sub-registers where possible, to use less memory when possible, like changing `mov rax, 1` to `mov eax,0x1`.

If we wanted to only show the assembly code, without machine code or addresses, we could add the `--no-show-raw-insn --no-addresses` flags, as follows:

shellsession

```
kimvutrinh@htb[/htb]$ objdump -M intel --no-show-raw-insn --no-addresses -d  
helloWorld
```

```
helloWorld:      file format elf64-x86-64
```

```
Disassembly of section .text:
```

```
<_start>:  
    mov     eax,0x1  
    mov     edi,0x1  
    movabs  rsi,0x402000  
    mov     edx,0x12  
    syscall  
    mov     eax,0x3c  
    mov     edi,0x0  
    syscall
```

Note: Note that `objdump` has changed the third instruction into `movabs`. This is the same as `mov`, so in case you need to reassemble the code, you can change it back to `mov`.

The `-d` flag will only disassemble the `.text` section of our code. To dump any strings, we can use the `-s` flag, and add `-j .data` to only examine the `.data` section. This means that we also do not need to add `-M intel`. The final command is as follows:

```
kimvutrinh@htb[/htb]$ objdump -sj .data helloWorld
```

```
helloWorld:      file format elf64-x86-64
```

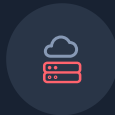
```
Contents of section .data:
```

shellsession

```
402000 48656c6c 6f204854 42204163 6164656d Hello HTB Academ
402010 7921 y!
```

As we can see, the `.data` section indeed contains the `message` variable with the string `Hello HTB Academy!`. This should give us a better idea of how our code was assembled into machine code and how it looks after we assemble it. Next, let us go through the basics of code debugging, which is a critical skill we need to learn.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions ⓘ

PRO

⚠ Answer questions in English to ensure an accurate response.

Question 1

 +1

 +40



Download the attached file and disassemble it to find the flag

Write your answer

 Submit

 Hint

 Download File

← Previous

Section 7 / 24

XP +20

Next

Mark Complete & Next